MCQ-4

Python Programming

Name: _____ **Date:** _____ **Score:** _____

Instructions: Select the correct option by filling or marking the checkbox ☐ next to it. Read each code snippet carefully — several questions are intentionally tricky.

1. What is printed by the following code?

```
x = 5
y = "5"
print(x == y, x is y)
```

   ☐ False False
   ☐ True False
   ☐ True True
   ☐ False True

2. What is the output of this for-else loop?

```
for i in range(3):
    if i == 5:
        break
else:
    print("done")
```

   ☐ Nothing is printed
   ☐ done
   ☐ An error occurs because else needs an if
   ☐ It loops forever

3. What is printed by the following code?

```
def add_item(item, lst=[]):
    lst.append(item)
    return lst
print(add_item(1))
print(add_item(2))
```

   ☐ [1] [2]
   ☐ [1] [1, 2]
   ☐ [1, 2] [1, 2]
   ☐ TypeError: mutable default argument

4. What is the output of the following slicing operation?

```
s = "Hello, World!"
print(s[-6:-1])
```

- ☐ World
- ☐ World!
- ☐ Worl
- ☐ orld!

5. What is the output of this dictionary comprehension?

```
d = {x: x**2 for x in range(5) if x % 2 == 0}
print(d)
```

- ☐ {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
- ☐ {0: 0, 2: 4, 4: 16}
- ☐ {1: 1, 3: 9}
- ☐ SyntaxError

6. What happens when the following code runs?

```
count = 0
def increment():
    count += 1
    return count
print(increment())
```

- ☐ Prints 1
- ☐ Prints 0
- ☐ UnboundLocalError: local variable referenced before assignment
- ☐ NameError: count is not defined

7. What is the output of the following expression?

```
print(True + True + True == 3)
```

- ☐ True
- ☐ False
- ☐ TypeError: unsupported operand types
- ☐ 3 == 3 is never evaluated

8. What is printed by the following code?

```
a = 256
b = 256
c = 257
d = 257
print(a is b, c is d)
```

- ☐ True True
- ☐ False False
- ☐ True False
- ☐ False True

9. What is the output of the following code?

```
class Counter:
    count = 0
    def __init__(self):
        Counter.count += 1
c1 = Counter()
c2 = Counter()
print(c1.count, c2.count, Counter.count)
```

- ☐ 1 1 2
- ☐ 2 2 2
- ☐ 1 2 2
- ☐ 0 0 2

10. What does the following code print?

```
try:
    raise ValueError("bad")
except (TypeError, ValueError) as e:
    print("caught:", e)
except Exception:
    print("generic")
```

- ☐ generic
- ☐ caught: bad
- ☐ Both except blocks run
- ☐ SyntaxError: multiple except types not allowed

11. What is the output of the following generator code?

```
def gen():
    yield 1
    yield 2
    yield 3
g = gen()
print(next(g), next(g))
print(sum(g))
```

- ☐ 1 2 3
- ☐ 1 2 6
- ☐ 1 2 5
- ☐ StopIteration is raised immediately

12. What is the output of the following unpacking operation?

```
a, *b, c = [1, 2, 3, 4, 5]
print(a, b, c)
```

- ☐ 1 [2, 3, 4] 5
- ☐ 1 2 5

- ☐ 1 [2, 3, 4, 5] 5
- ☐ ValueError: too many values to unpack

13. What is printed by the following code?

```
def logger(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
@logger
def greet():
    """Say hello"""
    print("hi")
print(greet.__name__)
```

- ☐ greet
- ☐ wrapper
- ☐ logger
- ☐ AttributeError: function has no __name__

14. Which file mode opens a file for both reading and writing, truncating (emptying) it if it already exists?

- ☐ "r+"
- ☐ "a+"
- ☐ "w+"
- ☐ "x+"

15. What is printed by the following code?

```
try:
    with open("data.txt", "w") as f:
        f.write("Hello")
        raise ValueError("oops")
except ValueError:
    pass
print(f.closed)
```

- ☐ False
- ☐ True
- ☐ AttributeError: f is not defined outside the with block
- ☐ The program crashes before reaching print()

16. Assuming data.txt contains several lines of text, what is printed?

```
with open("data.txt") as f:
    lines1 = f.readlines()
    lines2 = f.readlines()
print(lines2)
```

- ☐ The same list of lines as lines1
- ☐ []

☐ None
☐ ValueError: I/O operation on closed file

17. What is the output of the following set operation?

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a ^ b)
```

☐ {2, 3}
☐ {1, 2, 3, 4}
☐ {1, 4}
☐ {1}

18. What is the output of the following code?

```
data = [(1, 'b'), (2, 'a'), (3, 'c')]
print(sorted(data, key=lambda x: x[1]))
```

☐ [(1, 'b'), (2, 'a'), (3, 'c')]
☐ [(2, 'a'), (1, 'b'), (3, 'c')]
☐ [(3, 'c'), (2, 'a'), (1, 'b')]
☐ TypeError: lambda cannot be used as a sort key

19. What is the output of the following code?

```
class A:
    def who(self): print("A")
class B(A):
    pass
class C(A):
    def who(self): print("C")
class D(B, C):
    pass
D().who()
```

☐ A
☐ B
☐ C
☐ TypeError: cannot create consistent MRO

20. What is the output of the following code?

```
class CustomError(Exception):
    pass
try:
    try:
        raise ValueError("inner")
    except ValueError as e:
        raise CustomError("outer") from e
except CustomError as ce:
    print(ce.__cause__)
```

☐ outer
☐ inner